Lecture 8 · Mon 10/26/2026

CS160 Compilers — Fall 2026

Functions and the Machine.

Yu Feng

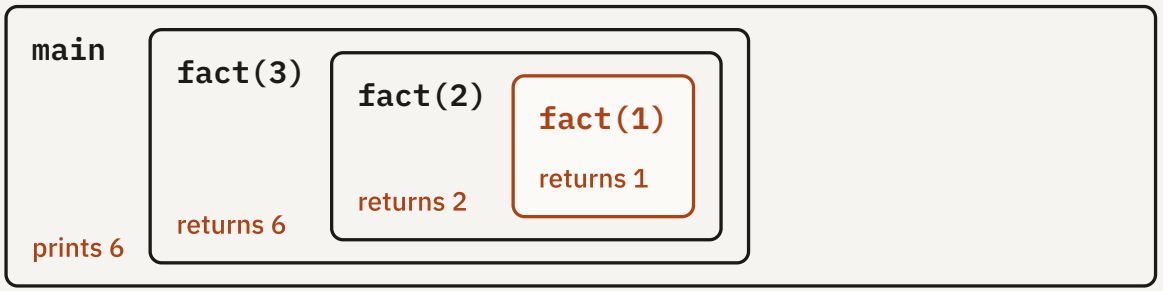
What happens when `fact(3)` runs.

fact.py · verified

```
def fact(n: int) -> int:
    if n <= 1:
        return 1
    return n * fact(n - 1)

print(fact(3))
```

the calls, nested left to right

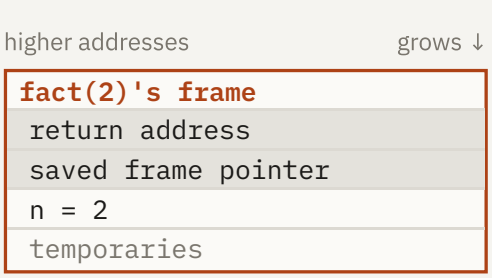


activation

one execution of a function body, from the call to its return. `fact` is one function; while `fact(1)` runs, three activations of it exist at once.

Each activation has its own n, and each has to remember where to continue when the call it made comes back.

The activation record.



activation record, also frame

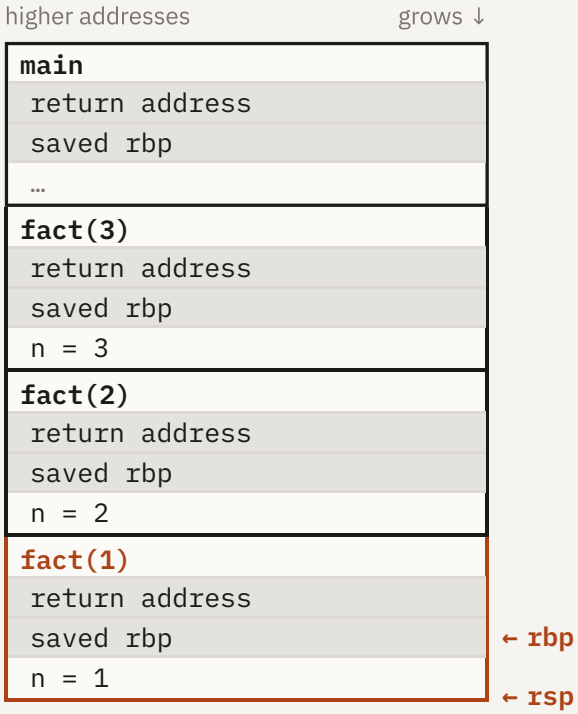
the block of memory holding one activation's data: where to go back to, the caller's frame pointer, the parameters and locals

return address

the address of the instruction right after the call — where execution continues when this activation returns.

Lecture 3's `alloca` slots were this frame's locals. Each *activation* gets its own frame, so the three running `facts` have three separate ns.

Frames form a stack.



A call pushes a frame below the current one; a return pops it. Activations end in the reverse of the order they began — a **stack**.

Two registers track it: `rsp` points at the bottom of the current frame, `rbp` at its top; every slot is at a fixed distance from `rbp`. The stack grows toward *lower* addresses.

In LLVM: `define`, `call`, `ret`.

```
define i32 @fact(i32 %n) {              ; a function with a typed parameter
entry:
  %n.addr = alloca i32
  store i32 %n, ptr %n.addr             ; the parameter gets a slot, like any variable
  ...
  ret i32 %t11                          ; typed return
}

  %t1 = call i32 @fact(i32 3)           ; a typed call, in main
```

We say *what* is called with *which* values and *what* comes back; where the arguments go, where the frame is, how the return address is saved are left to clang (slides 10–14).

Compiling a call.

compile_expr · cocoa8.py

```
case ast.Call(func=ast.Name(id=f), args=args):          # f(e1, ..., en)
    vals = [compile_expr(a, out) for a in args]         # evaluate the arguments, left to right
    t = "%" + out.fresh()
    out.emit(f"{t} = call i32 @{f}({', '.join('i32 ' + v for v in vals)})")
    return t
```

A call is an expression: it has a value, so it returns a name like every other case.

Compiling `def` and `return`.

compile_function · trimmed

```
out = Emitter(globals_, in_function=True)        # a fresh environment for this function
for p in params:                                 # each parameter gets its own slot
    out.env[p] = f"%{p}.addr"
    out.emit(f"%{p}.addr = alloca i32")
    out.emit(f"store i32 %{p}, ptr %{p}.addr")
for st in fd.body: compile_stmt(st, out)
out.emit("ret i32 0")                            # in case the body falls off the end

case ast.Return(value=v):
    out.emit(f"ret i32 {compile_expr(v, out)}")
    out.label(out.fresh("after"))                # ret ends a chunk; start a new one
```

The `after` label: Lecture 3's rule that a chunk ends at a jump — `ret` is one, and an `if` is about to emit `br label %end1` right after it.

Our compiler · 4/5

Scopes: locals first, then globals.

scope.py · verified · prints 11 then 21

```
step: int = 10

def bump(x: int) -> int:
    y: int = x + step      # step is global, y is local
    return y

print(bump(1))
print(bump(bump(1)))
```

scope

the part of a program in which a name refers to one particular variable. A fresh `Emitter` per function is a fresh scope: its `env` starts empty, so this function's `x` is not anyone else's.

`Emitter.lookup(x)`: env first, then `globals`, else error. Top-level variables become `@step = global i32 0`; function locals stay `allocas`.

Our compiler · 5/5

The IR for `fact`.

python3 cocoa8.py fact.py · verified · trimmed

```
define i32 @fact(i32 %n) {
entry:
  %n.addr = alloca i32
  store i32 %n, ptr %n.addr
  %t1 = load i32, ptr %n.addr
  %t2 = icmp sle i32 %t1, 1
  br i1 %t2, label %then1, label %end1
then1:
  ret i32 1
end1:
  %t7 = load i32, ptr %n.addr              ; n, loaded before the call
  %t8 = load i32, ptr %n.addr
  %t9 = sub i32 %t8, 1
  %t10 = call i32 @fact(i32 %t9)
  %t11 = mul i32 %t7, %t10                 ; n, used after the call
  ret i32 %t11
}
```

`%n.addr = alloca i32` runs on *every* entry, in a new frame, so `fact(3)`'s n and `fact(2)`'s n are different slots (slide 3).

`%t7` is loaded before the call and used after it.

Why look at the machine at all.

calling convention

the agreement every function on this machine follows: which registers carry the arguments, which carries the result, who saves what. On x86-64 Linux and macOS it is the *System V* convention. clang follows it for every `define` and `call` we emit.

We will never write assembly. clang turns slide 5's three instructions into everything slides 2–4 described, and keeps the convention — `print_int` was compiled months ago and cooperates with our code only because both sides follow it.

We look once, for three things: how three activations keep three ns (the frame, slide 12); why a value that is needed after a call must be put in memory first (a *spill*, slide 14 — the start of register allocation in Lecture 15); why a local cannot outlive its function.

Under clang's hood · 2/5

The calling sequence.

caller
the code making the call

callee
the function being called

calling sequence
the fixed steps that together push a frame on the way in and pop it on the way out.

caller, before the call:	put the arguments where the callee expects them save the return address; jump to the callee (one instruction: call)
callee, on entry:	save the caller's frame pointer; set up its own frame
callee, before returning:	put the result where the caller expects it restore the caller's frame pointer; pop the frame; jump to the return address (ret)
caller, after the call:	pick up the result; carry on

The caller and the callee are compiled separately; both follow these steps. Slide 12 shows them in clang's output.

The callee and the caller, in assembly.

`int add_mul(int a, int b) { int c = a * b; return c + a; }`

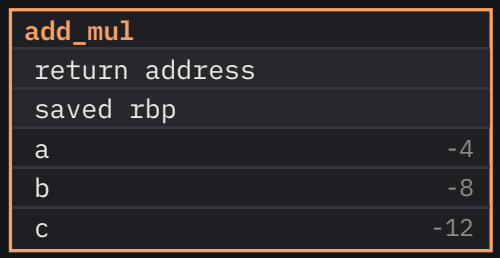
```
add_mul:
  pushq  %rbp             # save caller's rbp
  movq   %rsp, %rbp       # my frame starts here
  movl   %edi, -4(%rbp)   # a arrived in edi
  movl   %esi, -8(%rbp)   # b arrived in esi
  movl   -4(%rbp), %eax
  imull  -8(%rbp), %eax
  movl   %eax, -12(%rbp)  # c's slot
  movl   -12(%rbp), %eax
  addl   -4(%rbp), %eax   # result -> eax
  popq   %rbp             # restore caller's rbp
  retq                    # pop return address, jump
```

`int g(int x) { return add_mul(x, 7) + x; }`

```
g:
  pushq  %rbp
  movq   %rsp, %rbp
  subq   $16, %rsp
  movl   %edi, -4(%rbp)  # my x, into my frame
  movl   -4(%rbp), %edi  # 1st argument -> edi
  movl   $7, %esi        # 2nd argument -> esi
  callq  add_mul         # push return address, jump
  addl   -4(%rbp), %eax  # result came back in eax
  addq   $16, %rsp
  popq   %rbp
  retq
```

```
movq a, b  a → b
%r $7      register · literal
q l        64-bit · 32-bit
-8(%rbp)   memory at rbp − 8
```

higher addresses — grows ↓



Every line is one of slide 11's six steps. `-4`, `-8`, `-12` off `rbp` are Lecture 3's `allocas`: the frame is those slots plus two bookkeeping words. `callq` pushes the return address and jumps; `retq` pops it and jumps back.

Under clang's hood · 4/5

The register table, and who saves what.

arguments, in order	result	callee-saved	caller-saved
`rdi rsi rdx rcx r8 r9`	`rax`	`rbx rbp r12–r15`	**`everything else: rax rdi rsi rdx rcx r8–r11`**

caller-saved

the callee may overwrite it freely; if the caller still needs the value afterwards, the *caller* must save it before the call

callee-saved

the callee may use it only if it puts the old value back before returning.

That is why g on slide 12 copied x into its frame before the call and read it back after: x arrived in `rdi`, caller-saved, and `add_mul` may destroy it. The two right-hand columns say *whose job* it is to keep a value alive across a call.

Under clang's hood · 5/5

clang spills n in our own `fact`.

clang -O0 -S fact.ll · the end1 chunk of slide 9 · verified

```
movl   4(%rsp), %eax  # %t7 = n
movl   %eax, (%rsp)   # 4-byte Spill        ← clang's own comment
movl   4(%rsp), %edi
subl   $1, %edi
callq  fact@PLT
movl   %eax, %ecx
movl   (%rsp), %eax   # 4-byte Reload
imull  %ecx, %eax
retq
```

`movq a, b`	a → b
`%r $7`	register · literal
`q l`	64-bit · 32-bit
`-8(%rbp)`	memory at rbp − 8

higher addresses — grows ↓

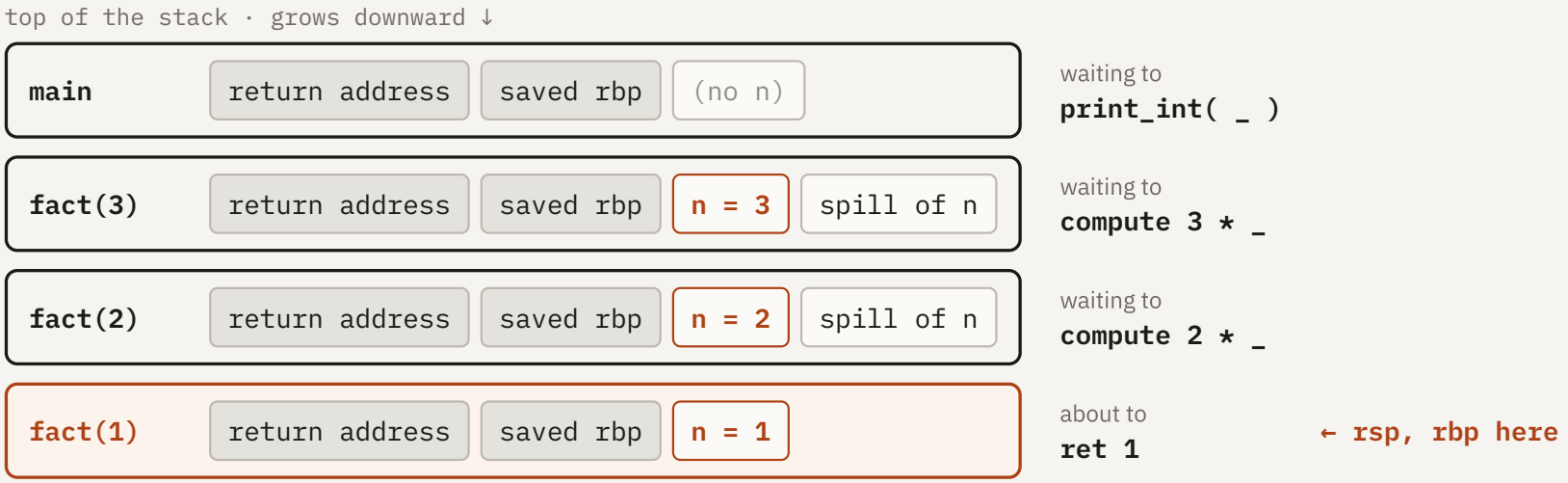


spill

storing a value that was in a register into a frame slot, because the register cannot be trusted to keep it — here, across a call.

`%t7` sat in `rax`: caller-saved, and the register the result comes back in. It is needed after the call, so clang stored it to the frame and reloaded it (slide 13's rule). `@PLT`: read as plain `fact`.

The stack at the deepest point of `fact(3)`.



Four frames, one per activation still running; each has its own n, and its own copy of n kept across the call (slide 14). Then it unwinds from the bottom: `fact(1)` returns 1, `fact(2)` computes 2, `fact(3)` computes 6, `main` prints it — each `ret` pops one frame.

Wednesday; PA3.

One thing nothing checked today: `print(f(1, 2))` with `def f(a: int)` compiles, links and prints 1 (verified). clang checks the IR, not ChocoPy.

Wednesday: semantic analysis — a pass that rejects it with a line number; types as rules on paper, `O, R ⊢ e : T`.

PA3 (out, due Fri 10/30): functions — `def`, `return`, calls — plus Wednesday's checker. Slides 6–7 are the codegen half. Thursday's section is the PA3 lab.

